

Professionell Code Review i AI-Projekt

Teamarbete, PR-flöden & Kvalitetssäkring (90 minuter)

Scenario: Ni är två AI-utvecklare i samma ML-team

VAD?

Praktisk Code Review & Pull Requests

VARFÖR?

Kvalitetssäkra AI-kod & undvika buggar

HUR?

1+1 Review & Feedback-loop i GitHub

GitHub

<https://github.com/Ars1222022/git-workshop.git>

Systemdesign: Vad gör filerna i vårt AI-repo och varför?

main.py — Systemets Huvudmotor

Kopplar samman dataladdning och modellutvärdering. Kör hela pipeline från start till mål.

data_loader.py — Data Ingestion & Sanering

Läser in kunddata (churn). Ansvarar för att validera och rensa korrupt indata.

model_evaluator.py — AI-modellens Utvärdering

Beräknar klassificeringsmått (Accuracy, Precision, Recall) och hanterar edge cases.

tests/test_evaluator.py — Automatiserade Pytest-tester

Garanterar att systemets alla moduler fungerar och stoppar trasig kod från att mergas.

Git/GitHub Grundflöde: Från Repo till Feature Branch

0. Kom igång (EN GANG)

1. GitHub-konto: github.com -> Sign up (gratis)
2. Berätta vem du är mot Git:
`git config --global user.name "Ditt Namn"`
`git config --global user.email "din@epost.se"`
3. Kopiera projektet: python workshop_setup.py
(skapar ai-system/ och installerar pytest)

1. Skapa Repo & Bjud in

DEV A: github.com -> New Repository -> ai-system
(kryssa i Add a README) -> Create
DEV A: repot -> Settings -> Collaborators
-> Add people -> DEV B:s användarnamn
DEV B: acceptera inbjudan (e-post)
DEV B: `git clone <URL>` # laddar ner repot

2. Skapa Feature Branch

Arbeta ALDRIG direkt på main.
Branch = en egen kopia att testa i.
`git checkout -b feature/data-validation`
Kolla var du är: `git branch`

3. Spara & Skicka

Varje gång du färdigställt kod:
`git add .`
`git commit -m "feat: add validation"`
`git push origin feature/data-validation`
Nu är din ändring på GitHub - redo för PR.

Pull Request & 1+1 Code Review: Kvalitetssäkra Koden

Skapa Pull Request (PR)

- VAD? En begäran om att slå ihop din branch med main.
- STEG 1: Skriv PR-beskrivningen själv först!
 - Vad har du ändrat?
 - Varför behövs ändringen?
- STEG 2: Använd AI som assistent!
 - Klistra in din text i Copilot/AI för språkkontroll och strukturering.
 - Låt AI kontrollera att du inte glömt viktiga detaljer.
- WORKSHOPREGEL:
Alla PRs måste godkännas av en kollega innan merge!

1+1 Code Review-Regeln

När du granskar din kollegas PR ska du ge:

1. [+] Beröm (1 sak som är bra)
 - T.ex. 'Snygg felhantering!', 'Tydliga docstrings!'
2. [!] Förbättringspunkt (1 konkret förslag)
 - T.ex. 'Vad händer om listan är tom?', 'Lägg till ett pytest-fall för detta edge-case.'
3. Reviewerens 3 Linser:
 - Korrekthet (Fungerar logiken?)
 - Läsbarhet (Är koden enkel att förstå?)
 - Tester (Finns relevanta pytest-fall?)

Feedback-loopen & Squash Merge (Workshopens Hjärta)

Hur fungerar feedback-loopen i praktiken?

1. Utvecklaren får feedback i sin PR från Reviewern.
2. Utvecklaren ändrar koden lokalt i VS Code och gör 'git commit' + 'git push'.
3. GitHub uppdaterar automatiskt PR:n med den nya committen!
4. Reviewern ser ändringen, blir nöjd och trycker 'Approve'.

Synka med main & Konflikter

- Om main har uppdaterats under tiden:
Kör 'git checkout main' -> 'git pull' -> switcha tillbaka och kör 'git merge main'.
- Om konflikt uppstår i VS Code:
Välj 'Accept Current' eller 'Incoming', spara, commit och push!

Squash and Merge

- VAD? Slår ihop alla små-commits från din branch till EN ren commit på main.
- VARFÖR? Håller main-historiken städad och lättläst.
- HUR? Välj 'Squash and Merge' i dropdown-menyn på GitHub när PR:n godkännts.

Workshop Del 1: Rollfördelning & Kodändringar (25 min)

Dev A: Indatavalidering

- UPPGIFT: Filtrera bort korrupt data i `data_loader.py`.
- VAD SKA GÖRAS?
 1. Skapa branch 'feature/add-data-validation'.
 2. Rensa bort poster där `tenure_months < 0`.
 3. Skriv ett pytest-fall i `tests/test_evaluator.py` som verifierar att inga negativa värden finns kvar.
 4. Commit & push.
 5. Skriv PR-text manuellt -> förbättra med AI -> öppna PR!

Dev B: Precision & Recall

- UPPGIFT: Utöka utvärderingsmått i `model_evaluator.py`.
- VAD SKA GÖRAS?
 1. Skapa branch 'feature/add-precision-recall'.
 2. Implementera beräkning av precision och recall (med nolldivisionsskydd!).
 3. Skriv ett pytest-fall i `tests/test_evaluator.py` för precision/recall.
 4. Commit & push.
 5. Skriv PR-text manuellt -> förbättra med AI -> öppna PR!

Workshop Del 2: Code Review, Feedback & Roller (40 min)

Runda 1: Dev A granskar Dev B (20 min)

1. Dev A öppnar Dev B:s PR på GitHub.
2. Dev A tillämpar 1+1-regeln:
 - [+] 1 Beröm i en kommentar.
 - [!] 1 Förbättringspunkt (t.ex. 'Vad händer om `tp+fp == 0`?').
3. Dev B åtgärdar koden lokalt i VS Code, commit & push.
4. Dev A ser uppdateringen, klickar 'Approve' och 'Squash and Merge'!

Runda 2: Byt roller! Dev B granskar Dev A (20 min)

1. Dev B öppnar Dev A:s PR på GitHub.
2. Dev B tillämpar 1+1-regeln:
 - [+] 1 Beröm i en kommentar.
 - [!] 1 Förbättringspunkt (t.ex. 'Lägg till ett testfall i pytest som kollar att längden minskat').
3. Dev A åtgärdar koden lokalt i VS Code, commit & push.
4. Dev B ser uppdateringen, klickar 'Approve' och 'Squash and Merge'!

Workshop Del 3: Integration, Pytest & Utvärdering (15 min)

Slutintegration på main (10 min)

1. Båda utvecklarna kör lokalt i terminalen: `git checkout main -> git pull origin main`.
2. Verifiera att hela systemet fungerar med båda förbättringarna: `python main.py`.
3. Kör alla automatiserade tester: `pytest`.

Utvärdering & Lärdomar (5 min)

Hur kändes det att ge och ta emot 1+1-feedback?
Vad gjorde PR-beskrivningen bra?
Vad hade du velat veta redan i steg 0?